

Simulator Specification — `drone_dispatch_env` (Revision 2)

Software Definition and Requirements (Instructor / TA build, student API contract)

This document defines the simulator that the course term project runs against. It is the API contract: students may rely on everything specified here and must not depend on internals beyond it. Build target is a **lightweight 2-D, Tier-A operational simulator** — deliberately *not* a 3-D, weather-calibrated city model. The cost is in correctness and a clean interface, not in fidelity.

Revision 2 note. This revision folds in conventions that the original spec left implicit or stated inconsistently, and that were pinned and verified during the build (action layout, stepping semantics, observation shapes, the meaning of `K_max`, grid connectivity, the offline format). Changed/clarified items are marked **[R2]**. Section 12 is a new, normative “frozen API conventions” appendix — that section is the authoritative contract for student code.

1. Scope and Non-Goals

In scope (operational layer): order-to-drone assignment, battery/charging management, idle-drone repositioning, on a fixed grid with fixed hubs and fixed fleet.

Out of scope (fixed constants, not decisions): hub placement, fleet size/composition, airspace design, 3-D flight dynamics, weather, real geographic calibration. These are parameters, not actions.

Two required deliverables: (1) the gymnasium-compatible environment (`reset`, `step`, `render`, `observation_space`, `action_space`, seeding via `reset(seed=...)`), and (2) a **visualizer + episode replayer** (Section 8). The visualizer is treated as essential, not optional — it is the main tool through which students understand and debug what their agents do.

2. World Model

- **Grid:** $H \times W$ cells (default 20×20), configurable. Each cell is free, a **no-fly** cell, or a **hub** (chargers are co-located at hubs). **[R2]** Because chargers are always co-located with hubs, a hub cell and a charger cell are the same cell; in the encoded grid they share a single code (see Section 12.3). **[R2]** No-fly placement is **connectivity-preserving**: the free space (all non-no-fly cells) is always fully connected, so every hub, pickup, dropoff, and charger is reachable. Students may rely on this even under high-no-fly stress configs.
- **Time:** discrete steps $t = 0 \dots T_{\max}$ (default $T_{\max} = 500$).
- **Drones:** N drones (default 8), each with:
 - position (x, y) , integer cell;
 - battery state of charge `soc` $[0, 1]$;
 - status $\{\text{idle, to_pickup, to_dropoff, to_charger, charging}\}$;
 - current assigned order id (or none);
 - payload class capacity (single class by default).
- **Orders:** arrive by a **Poisson process** (rate per step, default 0.3). Each order has: origin cell, destination cell, creation time, **deadline** (creation + `sla_steps`, default 60), weight (single class by default). Unserved orders past deadline are **dropped** (penalized). **[R2]** Orders accumulate without a hard cap; `K_max` is an *observation/decision window*, not a limit on how many orders may exist (see Section 12.2). Overflow demand beyond the window still ages and is dropped on deadline — it is never silently discarded.
- **Movement:** one cell per step in the 4- or 8-neighborhood (configurable; default 4). Entering a no-fly cell is **forbidden** (masked out / hard-rejected). Low-level pathing is a no-fly-aware shortest-path (BFS) router.

- **Battery:** each move consumes `e_move` SoC (default 0.01); hovering/idle consumes `e_idle` (default 0.001). Charging at a charger restores `c_rate` SoC/step (default 0.05). If `soc` hits 0 while airborne → **depletion event** (drone lost for the episode, large penalty). **[R2]** On depletion the carried order (if any) is returned to the pool (un-picked) so it can be reassigned or expire normally — it is not orphaned.

3. Decision Modes

The package ships **three env classes** behind one installation. Students select via the env id.

3.1 DroneDispatch-v0 — centralized dispatcher (discrete)

The agent is a single dispatcher acting at **decision epochs**. **[R2]** A decision epoch is a state where **at least one drone is idle AND at least one assignable order is pending**; otherwise the env auto-advances time internally until the next epoch or termination.

- **Observation (Dict):** exact shapes and encodings in Section 12.1. Summary:
 - `drones`: `[N, 10]` — (`x`, `y`, `soc`, `alive`, `status_onehot[5]`, `has_order`) **[R2]** (the original “`[N,5]` with `status_onehot...`” was under-specified; the one-hot is expanded and an explicit `alive` flag is included).
 - `orders`: `[K_max, 5]` — (`origin_x`, `origin_y`, `dest_x`, `dest_y`, `age`), zero-padded. **[R2]** The window contains the `K_max` **most-urgent assignable** orders (smallest deadline first).
 - `grid`: `[H, W]` static map — provided every step; may be cached.
 - `time`: `[1]` scalar normalized `t / T_max`.
 - `action_mask`: `[n_actions]` — also mirrored in `info["action_mask"]`.
- **Action (discrete, masked):** `Discrete(N * K_max + N + 1)`. **[R2]** Index layout is frozen in Section 12.2: `assignment (idle_drone, order_slot)`, `send_to_charge(drone)`, or `no-op` (defer). Invalid actions are masked; selecting one (a method that ignores the mask) is treated as a **no-op with a small penalty** (it advances time, so episodes always terminate). > Pathing between cells is handled internally by the no-fly-aware router. The agent decides *assignment and charging*, not per-cell movement, in this mode.

3.2 DroneControl-v0 — single-drone continuous control (for DDPG)

One drone, one active delivery. The agent sets **continuous speed and heading** each step to reach a target while managing energy.

- **Observation:** `Box(7)` — (`dx`, `dy`, `soc`, `dist_to_target`, `heading`, `nearest_nofly_dx`, `nearest_nofly_dy`).
- **Action:** `Box(2)` — (`speed [0,1]`, `heading_delta [-1,1]`).
- **Reward:** progress toward target — energy cost — time penalty — large penalty for entering a no-fly cell or depleting.
- **[R2] Maturity:** kinematic physics (no inertia); functional but not tuned. Treat as a v0.9 track.

3.3 DroneDispatchMA-v0 — decentralized multi-agent (for Ch. 21)

Each drone is an agent (dict-keyed parallel API: `reset/step` exchange `{agent_id: value}` maps).

- **Per-agent observation:** local view — own (`x`, `y`, `soc`, `status`), nearest assignable order (relative), nearest other drone (relative), and a local grid patch. Identical shape across agents to support **parameter sharing**.
- **Per-agent action:** `Discrete(4)` — {accept nearest order, move toward assigned target, go charge, idle}. **[R2]** “Go charge” only interrupts a not-yet-picked assignment (returning it to the pool); a drone carrying a picked payload finishes first.
- Designed so IDQN and independent A2C drop in. **[R2] Maturity:** functional; reward shaping is basic. Treat as a v0.9 track.

4. Reward Function (configurable weights)

Per-step team/episode reward (centralized) or per-agent (MA), composed of:

Term	Default weight	Meaning
<code>r_delivered</code>	+10	order delivered
<code>r_ontime_bonus</code>	+5	delivered on or before deadline
<code>r_late_penalty</code>	-0.1 / step late	lateness (applied once at delivery)
<code>r_dropped</code>	-15	order expired undelivered
<code>r_energy</code>	-1.0 × SoC consumed	energy cost
<code>r_depletion</code>	-50	battery hit 0 airborne (drone lost)
<code>r_noop</code>	-0.01	small cost on defer/idle to discourage stalling

All weights live in the config; the **standard eval config** fixes them. The reward is the spec — changing a weight changes optimal behavior, and the live-modification task exploits exactly this.

[R2] Reward accrual timing (dispatch mode). Assignment and `send_to_charge` actions update state without advancing time and return reward 0 for that `step()`. Physical reward terms (energy, deliveries, drops, depletion) accrue on the time-advancing steps (`no-op` and internal auto-advance). The per-step decomposition is exposed in `info["reward_terms"]` (Section 12.4), which is what the reward-stream overlay reads.

5. Episode and Termination

- Episode **terminates** when all drones are lost; **truncates** at `t = T_max`.
- `reset(seed)` regenerates the order stream and initial drone positions deterministically from the seed. Drones start at hubs with full battery. **[R2] reset** auto-advances to the first decision epoch; episode metrics are zeroed at that point so they reflect only the agent-controlled window.
- `info` exposes per-episode metrics (Section 7) every step (a superset of “at termination”).

6. Built-in Baselines (shipped, importable)

Students must beat these; they are provided so nobody re-implements them and comparisons are fair.

- **random** — random valid action.
- **greedy_nearest** — assign each pending order to the nearest available drone with sufficient SoC; send below-threshold idle drones to charge. **This is the primary bar.**
- **milp_rolling** — rolling-horizon assignment via a max-cardinality, min-cost matching (PuLP/CBC), recomputed each decision epoch, with the same charge-low-drone guard. The strong classical baseline. **[R2]** Currently single-epoch matching (no multi-step look-ahead) and uses Manhattan distance as the cost proxy; see Section 11 for the planned routed-distance upgrade.

Each baseline implements the same `act(obs) -> action` interface as a student policy (the `Policy` protocol in `agent_interface.py`).

7. Evaluation Harness and Metrics

`evaluate(policy, config, seeds)` runs N episodes and returns mean + per-seed:

- **cost_per_order** (primary score) = total cost / delivered orders, where cost aggregates energy + lateness + drop + depletion penalties.
- delivery **success rate**, **on-time rate**
- mean **delivery time**
- **energy per delivered order**
- **depletion events** (safety; a direct count)
- **idle %**, **charger utilization**
- **episode_return** (raw summed reward, for convenience)

Standard eval config (`configs/eval_standard.yaml`) is released to students. **Held-out eval seeds and a held-out “stress” config** (higher , more no-fly cells, lower battery) are retained by the instructor and used for grading and the live task. `reproduce.sh` accepts overridable seed/config/policy so the held-out ones can be swapped in.

8. Visualization and Replay (required, instructor-built)

The visualizer is a **first-class deliverable**. For RL, watching a policy act is the fastest way to find bugs metrics hide (oscillation, charger-camping, ignored orders, reward gaming).

8.1 Live and replay rendering

- **Top-down grid animation:** drones colored by status, order markers colored by deadline urgency, no-fly zones, hubs/chargers, per-drone **battery bar** and short **motion trail**.
- **Episode recorder + player:** Recorder captures the full state sequence; **Replayer** supports **play / pause / step-forward / step-back / scrub** (interactive backend) and headless **GIF/MP4 export**.
- **Deterministic replay:** an episode is fully reconstructable from (**seed, config, policy**).
- **Headless mode:** rendering is decoupled from `step()`; cluster training with no display works unchanged.

8.2 RL-understanding overlays

- **Value-based:** per-action Q-values; state-value heatmap over the grid.
- **Policy-based:** action-probability distribution.
- **Reward stream:** live per-term decomposition (from `info["reward_terms"]`) — makes reward-gaming visible.
- **Assignment overlay:** chosen drone→order links (from `info["assignments"]`).
- **Offline / Ch. 20:** dataset-coverage heatmap vs. where the evaluated policy goes.
- **Multi-agent / Ch. 21:** per-agent local-view panels.

Agent-supplied overlays read through the frozen `agent_interface.Introspectable` hook (`action_values`, `action_probs`, `state_values`); any may return `None` to skip. Non-agent overlays read from `info` and from `D_logs`.

8.3 Comparison view

`compare(policy_a, policy_b, seed)` renders two policies on the **same seed** side by side (GIF or frame list).

8.4 Metrics dashboard

`metrics_dashboard(results)` plots `cost_per_order`, success/on-time rate, energy/order, depletion events; plus CSV/TensorBoard training-curve logging consumed from `logs/`.

8.5 Technical requirements

- Lightweight stack (matplotlib animation + imageio); no heavyweight engine.
 - **[R2]** Frame rendering uses the Gymnasium 1.0 convention: `render_mode` is set at construction and `render()` takes no arguments, returning an `rgb_array` frame. (The original `render(mode="rgb_array")` signature is pre-Gymnasium-1.0.)
 - Overlays stay algorithm-agnostic via the frozen `agent_interface.py`, which **must remain frozen** once students start.
-

9. Offline Dataset (`D_logs`) — for the offline-RL component (Ch. 20)

Shipped as a static `.npz` file. Generated by a **mixed behavior policy**: ~60% `greedy_nearest`, ~40% `noisy/-random`, over many seeded episodes.

- **[R2] Format:** arrays `observations`, `actions`, `rewards`, `next_observations`, `terminals`, `timeouts` (truncation flags), plus `episode_returns`. The `timeouts` array is required because episodes typically end by truncation (`t = T_max`), not true termination; episode boundaries = `terminals | timeouts`.
 - **Size:** 200k transitions (configurable).
 - **Documented coverage gaps** (intentional): some `(state, action)` regions are unvisited — the point of the exercise (naive offline Q-learning over-estimates there).
 - `load_offline_dataset()` returns the arrays; `make_preference_pairs()` yields trajectory-segment pairs labeled by return — **[R2]** segments are guaranteed to lie within a single episode (split on `terminals | timeouts`).
-

10. Reproducibility and Engineering Requirements

- Fully deterministic given a seed (`env`, order stream, initial state, recorded-episode replay).
 - No global mutable state across `reset` calls.
 - `render()` returns an `rgb_array` top-down view (see Section 8).
 - Pinned dependencies (`requirements.txt`); Python 3.10; `gymnasium`, `numpy`, `matplotlib`, `imageio`, and `PuLP` for the MILP baseline.
 - Installable as a package (`pip install -e .`) exposing the three `env` ids via `gymnasium.register`.
 - Unit tests for: no-fly enforcement, battery depletion, deadline drop, charging, action masking, deterministic reset, **[R2]** grid connectivity, overflow accounting, no orphaned orders, and observation-space containment.
-

11. Suggested Build Order / Remaining Work

Items 1–4, 6, 7, 9 (`env` + baselines + harness + visualizer + offline) are implemented and tested. Remaining/optional:

1. **[Recommended] Routed-distance assignment + cached BFS distance fields.** `greedy_nearest/milp_rolling` rank by Manhattan distance, mis-assigning around no-fly walls; precompute per-hub/per-target BFS distance maps once per reset and use true routed distances. Improves correctness *and* removes the main per-call BFS cost.
 2. Full rolling-horizon MILP (multi-step look-ahead over future arrivals/battery).
 3. Richer `DroneControl-v0` physics (inertia, finer heading) and tuned reward.
 4. `DroneDispatchMA-v0` reward shaping / credit assignment.
 5. **Learnability validation** — train a reference DQN on `DroneDispatch-v0` and confirm return climbs toward `greedy_nearest` before instructing students to beat the bar.
-

12. Frozen API Conventions (normative — code against this)

12.1 Observation (DroneDispatch-v0)

Dict with: - `drones`: float32 [N, 10], per drone [x, y, soc, alive, s0, s1, s2, s3, s4, has_order], where alive {0,1} (0 if lost), s0..s4 is a one-hot of status, has_order {0,1}. - `orders`: float32 [K_max, 5], per visible order [origin_x, origin_y, dest_x, dest_y, age], age = t - created; rows beyond the live count are zero. The window = the K_max smallest-deadline assignable (unpicked, unassigned) orders. - `grid`: int8 [H, W]. - `time`: float32 [1] = t / T_max. - `action_mask`: int8 [N*K_max + N + 1] (also in info).

12.2 Status codes and action-index layout

Status: idle=0, to_pickup=1, to_dropoff=2, to_charger=3, charging=4.

Action index a for Discrete(N*K_max + N + 1): - 0 ... N*K_max - 1 → **assign**: drone a // K_max, order slot a % K_max (slot indexes the orders window above). - N*K_max ... N*K_max + N - 1 → **send_to_charge**: drone a - N*K_max. - N*K_max + N (last) → **no-op** (defer).

Validity: an assignment is valid iff the drone is idle and the slot is a live order; **send_to_charge** iff the drone is idle and soc < 1; **no-op** always valid. Masked-out actions, if selected, act as no-op + r_noop.

12.3 Grid encoding

0 = free, 1 = no-fly, 3 = hub/charger. (Hubs and chargers are co-located and share code 3; code 2 is reserved/unused.) Free space is guaranteed connected.

12.4 info keys (every step/reset)

- `action_mask`: int8 [n_actions].
- `assignments`: list of (drone_index, order_id) chosen this step.
- `reward_terms`: dict with keys delivered, ontime, late, dropped, energy, depletion, noop (this step's contribution per term).
- `t`: current timestep.
- `metrics`: running per-episode metrics.

12.5 Stepping semantics (dispatch)

`step(assign|charge)` updates state, does **not** advance time, returns reward 0. `step(no-op)` (or a masked action) applies r_noop and advances one tick. After any action the env auto-advances ticks while it is not a decision epoch and not terminal. Decision epoch (1 idle non-lost drone) (1 assignable order).

12.6 Policy / Introspectable contract

Frozen in `agent_interface.py`. `Policy.act(obs) -> action` (required). `Introspectable.{action_values, action_probs, state_values}(obs) -> Optional[np.ndarray]` (optional; any may return None). Do not modify this file once students start.